



Recibe una cálida:

¡Bienvenida!

Te estábamos esperando 😊

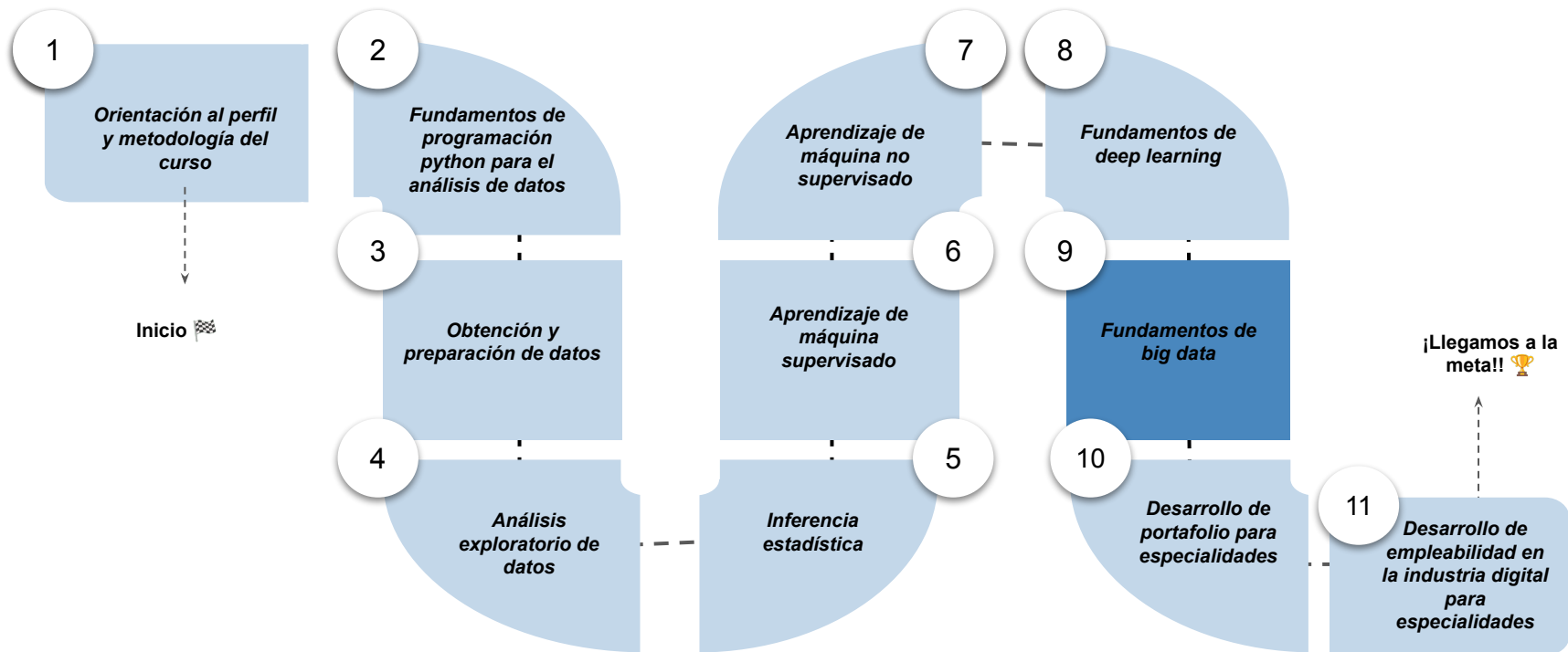


› Procesamiento de datos: estructurados

Aprendizaje Esperado 4: Manipular grandes volúmenes de datos utilizando spark sql para el procesamiento de grandes volúmenes de datos estructurados.

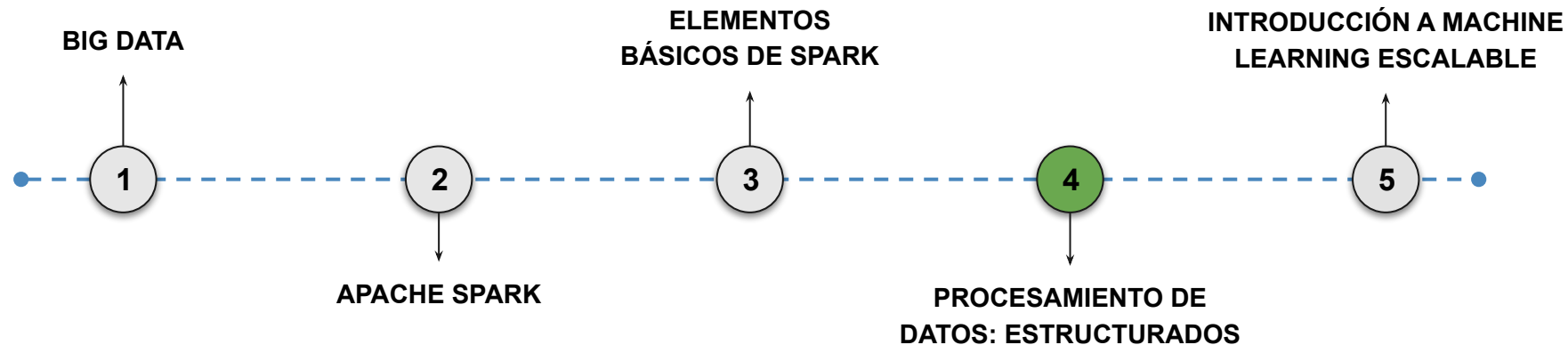
Hoja de ruta

¿Cuáles skills conforman el programa? **Fundamentos de Ciencia de Datos**



Roadmap de lecciones

¿Cuáles *lecciones* estaremos estudiando en este módulo?



Learning Path

¿Cuáles temas trabajaremos hoy?

4.

Procesamiento de datos estructurados con Spark SQL

Aprenderemos a utilizar Spark SQL y DataFrames para consultar, transformar y optimizar grandes volúmenes de datos estructurados de manera distribuida.

Introducción y fundamentos de Spark SQL

Qué es Spark SQL y sus beneficios.

Diferencias con el trabajo directo sobre RDD.

DataFrames y consultas SQL en Spark

Creación de DataFrames desde distintas fuentes.

Operaciones de filtrado, ordenamiento y agregación.

Learning Path

¿Cuáles temas trabajaremos hoy?

1.

Consultas avanzadas y optimización en Spark SQL

Aprenderemos a aplicar consultas y transformaciones avanzadas sobre DataFrames en Spark SQL, integrando funciones de agregación, uniones y expresiones SQL optimizadas mediante Catalyst y Tungsten.

Consultas y transformaciones avanzadas

Filtros y ordenamientos encadenados.

Agregaciones con funciones estadísticas.

Uso de selectExpr para expresiones SQL directas.

Optimización con Catalyst y Tungsten

Visualización y análisis del plan de ejecución optimizado.

Objetivos de aprendizaje

¿Qué aprenderás?

- Comprender el funcionamiento y ventajas de Spark SQL para datos estructurados.
- Crear y manipular DataFrames desde diversas fuentes de datos.
- Ejecutar consultas SQL en grandes volúmenes de datos distribuidos.
- Integrar datos de distintos formatos (CSV, JSON, Parquet) y sistemas (HDFS, S3, JDBC).
- Utilizar el motor Catalyst y el proyecto Tungsten para optimizar consultas.



Objetivos de aprendizaje

¿Qué aprenderás?



- Aplicar consultas y transformaciones avanzadas sobre DataFrames en Spark SQL.
- Utilizar funciones de agregación y unir múltiples DataFrames.
- Implementar expresiones SQL directamente en la API de DataFrame.
- Comprender el funcionamiento interno del Catalyst Optimizer y el proyecto Tungsten.
- Evaluar cómo estas optimizaciones impactan en el rendimiento de las consultas.

Procesamiento de datos: estructurados

Procesamiento de Datos Estructurados con Apache Spark

En los proyectos de análisis de datos, gran parte de la información que se procesa se encuentra en forma estructurada, es decir, organizada en tablas, con filas y columnas que representan registros y atributos. Este tipo de datos es habitual en bases de datos relacionales, hojas de cálculo, archivos CSV, JSON y Parquet, entre otros. Gestionar, consultar y transformar estos datos de forma eficiente es un desafío esencial en entornos de Big Data.

Introducción a Apache Spark

Apache Spark no solo ofrece capacidades avanzadas para el procesamiento distribuido de datos no estructurados, sino que también proporciona un potente módulo llamado **Spark SQL**, diseñado específicamente para trabajar con datos estructurados y semiestructurados.

Spark SQL amplía las capacidades de Spark, permitiendo la consulta de datos mediante sentencias SQL, la manipulación de **DataFrames** y la integración con diversas fuentes de datos.



Propósito de este manual

Comprensión de Spark SQL

Entender el funcionamiento de Spark SQL y sus componentes principales para el procesamiento de datos estructurados.

Ventajas de DataFrames

Conocer las ventajas de utilizar DataFrames para el manejo eficiente de datos estructurados.

Técnicas de optimización

Explorar las técnicas de optimización que permiten acelerar el procesamiento de consultas en entornos distribuidos.

Interoperabilidad y formatos

Abordar la interoperabilidad entre RDD y DataFrames, así como el uso de formatos de almacenamiento eficientes como JSON y Parquet.

Importancia del procesamiento de datos estructurados

El procesamiento de datos estructurados con Spark representa un recurso clave para analistas de datos, ingenieros y científicos de datos que necesitan manejar grandes volúmenes de información estructurada, integrando consultas SQL tradicionales con la potencia del procesamiento distribuido.

¿Qué es Spark SQL?

Spark SQL es un módulo de **Apache Spark** diseñado para trabajar con datos estructurados y semiestructurados. Ofrece una interfaz que permite realizar consultas utilizando el lenguaje SQL estándar, así como operaciones programáticas sobre estructuras de datos distribuidas llamadas **DataFrames** y **Datasets**.

Este módulo combina la familiaridad del lenguaje SQL con la potencia del procesamiento distribuido de Spark.

Evolución de Spark SQL

Antes de Spark SQL

Spark se centraba en el uso de **RDDs (Resilient Distributed Datasets)**, que ofrecían un alto nivel de flexibilidad pero requerían escribir un código más complejo para realizar consultas estructuradas.

Actualidad

Con Spark SQL, se simplificó el acceso y manipulación de datos estructurados, permitiendo ejecutar consultas SQL tradicionales sobre grandes conjuntos de datos distribuidos.

1

2

3

Introducción de Spark SQL

La incorporación de Spark SQL en el ecosistema Spark respondió a la necesidad de procesar datos estructurados de manera más eficiente y sencilla.

Beneficios principales de Spark SQL



Consultas SQL sobre datos distribuidos

Permite ejecutar consultas SQL tradicionales sobre grandes conjuntos de datos distribuidos, sin necesidad de mover los datos a un sistema gestor de bases de datos relacional (RDBMS).



API unificada

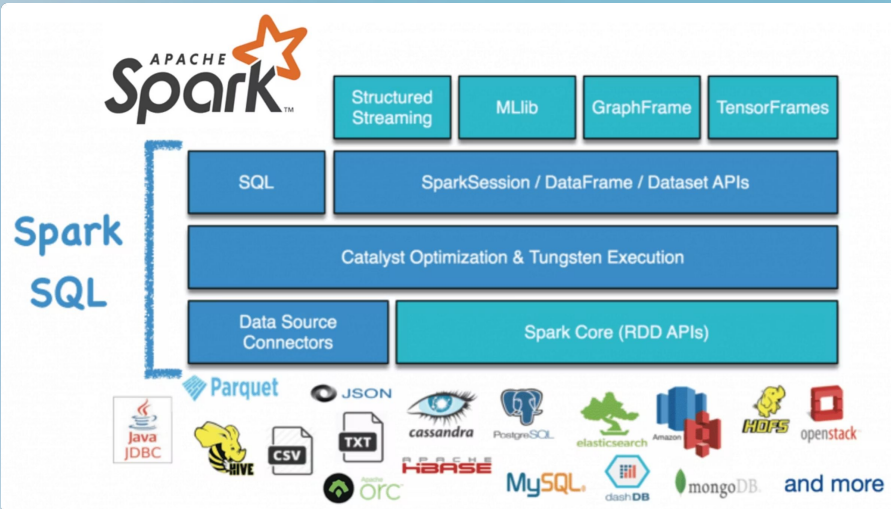
Proporciona una **API unificada** que permite a los usuarios trabajar con datos utilizando SQL, DataFrames y Datasets de forma intercambiable.



Soporte para múltiples fuentes de datos

Soporta múltiples fuentes de datos, incluyendo archivos **CSV, JSON, Parquet, ORC**, bases de datos JDBC y sistemas de almacenamiento distribuido como **HDFS, S3** y **Azure Blob Storage**.

Optimización en Spark SQL



Catalyst Optimizer

Utiliza un motor de consulta llamado **Catalyst Optimizer**, que analiza y mejora los planes de ejecución de las consultas para maximizar la eficiencia.

Proyecto Tungsten

Spark SQL aprovecha el proyecto **Tungsten** para optimizar el uso de la memoria y la CPU durante la ejecución.

Aplicaciones prácticas de Spark SQL



Limpieza de datos

Facilita la limpieza y preparación de grandes volúmenes de datos estructurados para su posterior análisis.



Generación de reportes

Facilita la creación de informes y visualizaciones a partir de grandes conjuntos de datos.



Integración de fuentes

Permite combinar datos provenientes de diferentes fuentes en un único análisis integrado.



Análisis exploratorio

Permite realizar análisis exploratorio de datos a gran escala y con altos niveles de rendimiento.

Características de Spark SQL

Spark SQL se ha consolidado como uno de los módulos más utilizados de Apache Spark, gracias a sus características que combinan flexibilidad, rendimiento y facilidad de uso. Estas características permiten a los desarrolladores y analistas de datos trabajar con grandes volúmenes de datos estructurados de manera eficiente.

Soporte para múltiples lenguajes



Python

Permite escribir código en Python utilizando la API de PySpark, ideal para científicos de datos y analistas.



Scala

Lenguaje nativo de Spark, ofrece el mejor rendimiento y acceso completo a todas las funcionalidades.



Java

Soporte completo para desarrollo en Java, ampliamente utilizado en entornos empresariales.



R

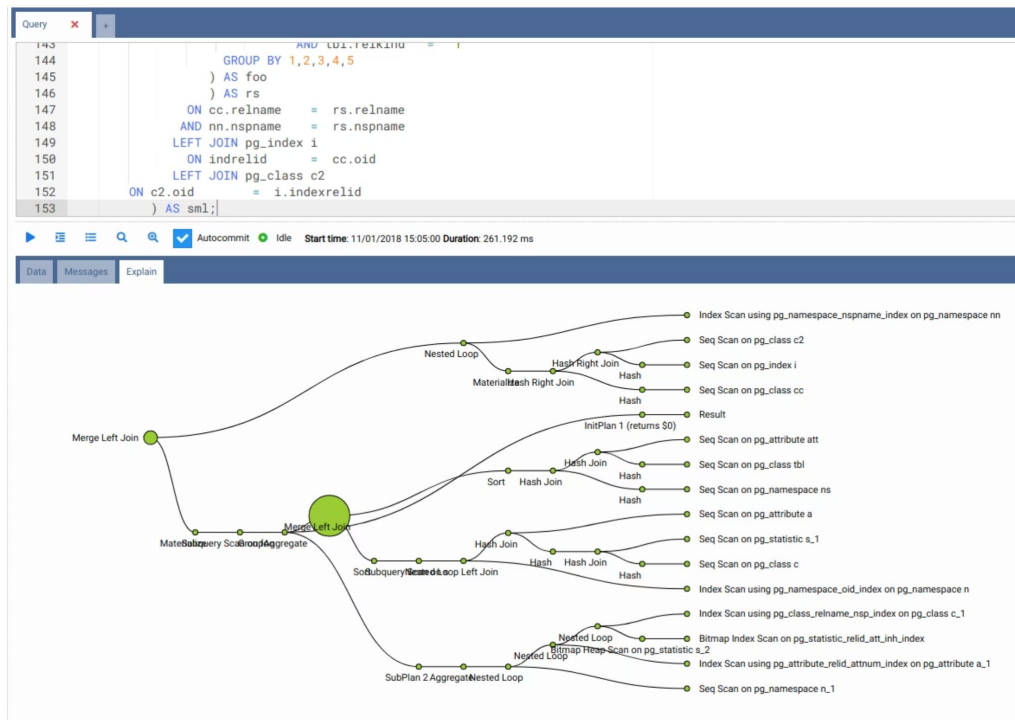
Integración con R para análisis estadísticos y visualizaciones avanzadas.

Esta diversidad de lenguajes facilita la adopción de Spark SQL en equipos multidisciplinarios con conocimientos en diferentes tecnologías.

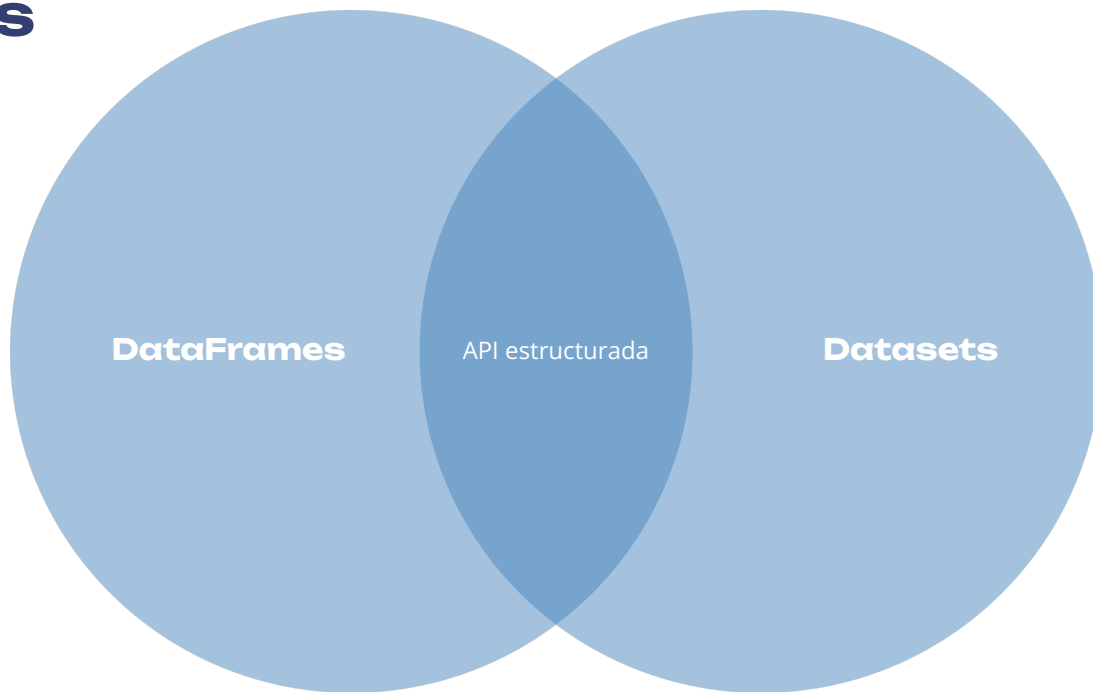
Consultas SQL tradicionales

Una característica importante es la posibilidad de ejecutar **consultas SQL tradicionales** sobre grandes volúmenes de datos.

Spark SQL permite realizar consultas complejas utilizando la sintaxis estándar de SQL, lo que resulta familiar para analistas y profesionales acostumbrados a trabajar con bases de datos relacionales.



DataFrames y Datasets



Spark SQL también destaca por su capacidad para trabajar con **DataFrames y Datasets**, estructuras de datos distribuidas que permiten manipular información estructurada de manera eficiente. Los DataFrames representan tablas con columnas y filas, similares a una hoja de cálculo o a una tabla en una base de datos.

Integración con fuentes de datos



CSV

Archivos de valores separados por comas, ampliamente utilizados para intercambio de datos tabulares.



JSON

Formato ligero de intercambio de datos, ideal para información semiestructurada.



JDBC

Conexión con bases de datos relacionales como MySQL, PostgreSQL y Oracle.



Almacenamiento distribuido

Integración con HDFS, Amazon S3 y Azure Blob Storage.

Motor de optimización Catalyst

Una característica clave de Spark SQL es su **motor de optimización Catalyst**. Este motor analiza las consultas y genera planes de ejecución optimizados que minimizan el movimiento de datos y aprovechan al máximo los recursos del clúster. Esto se traduce en una mejora significativa del rendimiento de las consultas.

Proyecto Tungsten



Gestión eficiente de memoria

Optimiza el uso de memoria mediante técnicas avanzadas de gestión y distribución.



Generación de código a bajo nivel

Genera código optimizado específico para cada consulta, mejorando el rendimiento.



Optimización de CPU

Aprovecha al máximo los recursos de procesamiento disponibles en el clúster.

Gracias a estas optimizaciones, Spark SQL es capaz de superar a muchos sistemas tradicionales de procesamiento de datos.

Consultas interactivas

Otra ventaja de Spark SQL es la posibilidad de realizar **consultas interactivas**. Los usuarios pueden conectarse a un clúster Spark y ejecutar consultas SQL en tiempo real, obteniendo resultados de manera rápida y eficiente.

¿Qué es una sesión Spark?

En el contexto de **Apache Spark**, la **sesión Spark** (o **SparkSession**) es el punto de entrada principal para interactuar con las funcionalidades de Spark, especialmente con las relacionadas al módulo **Spark SQL** y al manejo de datos estructurados mediante **DataFrames** y **Datasets**.

Evolución de la sesión Spark

Antes de Spark 2.0

Las aplicaciones Spark utilizaban un objeto llamado **SparkContext** para inicializar el entorno de ejecución.

1

2

Spark 2.0

Introducción de la clase `SparkSession` que unificó y simplificó la creación y configuración de las aplicaciones.

Actualidad

`SparkSession` integra tanto el `SparkContext` como el `SQLContext` en un solo objeto, facilitando el desarrollo de aplicaciones.

3

Creación de una sesión Spark

La creación de una sesión Spark es sencilla y requiere especificar algunos parámetros básicos, como el nombre de la aplicación y el modo de ejecución (local o en clúster).

Por ejemplo, en **PySpark**, se puede crear una sesión Spark de la siguiente manera:

```
from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("MiAplicacionSQL") \
    .master("local[*]") \
    .getOrCreate()
```

Acceso a componentes de la sesión Spark

SparkContext

Una vez creada la sesión, es posible acceder al **SparkContext** a través de sus atributos.

```
spark_context = spark.sparkContext
```

SQLContext

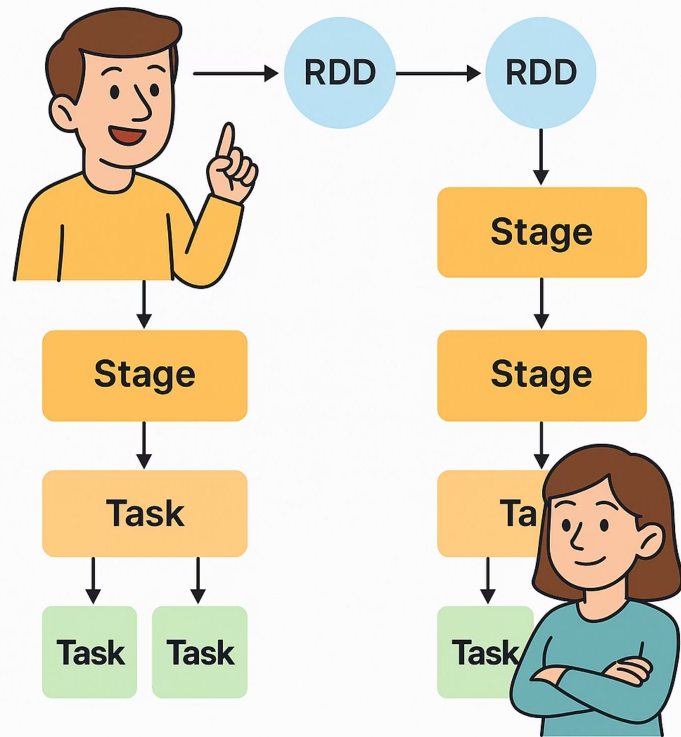
También se puede acceder al **SQLContext** para operaciones específicas de SQL.

```
sql_context = spark.sqlContext
```

Configuración de la sesión Spark

La sesión Spark también permite configurar parámetros importantes, como la cantidad de memoria asignada a los ejecutores, la cantidad de núcleos utilizados y la ubicación de los archivos de entrada y salida.

Basic of Spark Directed Acyclic Graph



Division of stages into tasks enables parallel processing

Estado de la aplicación

Otra ventaja de la sesión Spark es que mantiene el estado de la aplicación durante toda su ejecución. Esto significa que los usuarios pueden ejecutar múltiples consultas y transformaciones sobre los datos sin necesidad de reiniciar el contexto de ejecución.



Cierre de la sesión Spark

Es importante cerrar la sesión Spark al finalizar la aplicación para liberar los recursos asignados. Esto se realiza mediante el método `stop()`:

```
spark.stop()
```

¿Qué es un DataFrame Spark?

Uno de los elementos clave en el procesamiento de datos estructurados con **Apache Spark** es el concepto de **DataFrame**.

Un **DataFrame** es una colección distribuida de datos organizados en columnas, similar a una tabla en una base de datos relacional o a un DataFrame en Python (Pandas) o R.



Características de los DataFrames

Abstracción de alto nivel

Ofrece una abstracción de alto nivel que permite manipular y consultar datos estructurados de manera eficiente, utilizando tanto funciones de programación como consultas SQL.

Información estructural

Añade información estructural sobre las columnas y sus tipos de datos, lo que permite realizar optimizaciones adicionales mediante el motor **Catalyst**.

Soporte para diversos tipos

Puede contener datos de cualquier tipo, como números, cadenas de texto, fechas y estructuras más complejas como arrays y mapas.

Funcionamiento de los DataFrames

El funcionamiento de un DataFrame se basa en el principio de **evaluación perezosa** (lazy evaluation). Esto significa que las operaciones sobre un DataFrame no se ejecutan inmediatamente, sino que se registran como un plan lógico de ejecución. Este plan se optimiza y ejecuta únicamente cuando se solicita una acción que requiera un resultado, como `show()` o `count()`.

Ventajas de los DataFrames

Compatibilidad con múltiples lenguajes

Compatibilidad con múltiples lenguajes de programación, incluyendo **Python (PySpark)**, **Scala**, **Java** y **R**.

Operaciones avanzadas

Permiten realizar operaciones avanzadas sobre los datos, como filtrado, agregación, ordenamiento, unión y combinación de conjuntos de datos.

Interoperabilidad

Son interoperables con otros componentes de Spark, como Spark SQL, MLlib (biblioteca de aprendizaje automático) y GraphX (biblioteca de grafos).

Live Coding

¿En qué consistirá la Demo?

Exploraremos cómo crear una sesión Spark, generar DataFrames desde archivos y ejecutar consultas SQL distribuidas.

1. *Crear una SparkSession en PySpark.*
2. *Cargar datos desde CSV y JSON en DataFrames.*
3. *Registrar un DataFrame como vista temporal para usar SQL.*
4. *Ejecutar una consulta SQL con agregaciones y filtros.*
5. *Comparar el rendimiento con y sin optimizaciones Catalyst/Tungsten.*



Ejercicio

Consultas distribuidas con Spark SQL

Consultas distribuidas con Spark SQL

Contexto: 🙌

Aplicarás Spark SQL para realizar consultas complejas sobre grandes volúmenes de datos estructurados.

Consigna: 📝

1. Crea una sesión Spark y carga un conjunto de datos (CSV o Parquet).
2. Convierte los datos en un DataFrame y regístralo como vista temporal.
3. Escribe y ejecuta tres consultas SQL que incluyan filtros, agregaciones y ordenamientos.
4. Explica cómo Catalyst optimiza la ejecución de estas consultas.

Paso a paso: ⚙️

1. Configura la sesión Spark.
2. Carga los datos y crea el DataFrame.
3. Define las vistas temporales necesarias.
4. Ejecuta y analiza los resultados de las consultas.

Tiempo 🕒: 45 Minutos



Momento:

Time-out!



5 -10 min.



Procesamiento de datos: estructurados

Métodos de creación de un DataFrame

La creación de un **DataFrame** en Apache Spark es un proceso flexible que puede realizarse a partir de diferentes fuentes de datos y estructuras existentes. Spark proporciona diversos métodos para crear DataFrames, adaptándose a distintos escenarios y necesidades de análisis.

Spark – Different ways to create DataFrame

- Create DataFrame from RDD
 - `rdd.toDF()`
 - `spark.createDataFrame(rdd)`
- Create Directly from Data (Seq, List)
 - `data.toDF()`
 - `spark.createDataFrame(rdd)`
- Create from File Data sources
 - CSV, Parquet, ORC, Avro, JSON, Text files
- Create from Databases
 - Hive, HBase, MySQL, Oracle
- Create DataFrame from existing DataFrame or Dataset



Creación desde colecciones locales

Una de las formas más sencillas de crear un DataFrame es a partir de una colección de datos local, como una lista o un diccionario en Python.

```
# Crear una lista de tuplas
data = [("Juan", 30), ("Ana", 25), ("Carlos", 35)]

# Crear un DataFrame con nombres de columnas
df = spark.createDataFrame(data, ["nombre", "edad"])
```

Creación desde archivos CSV

Otra manera común de crear un DataFrame es mediante la lectura de archivos estructurados. Spark SQL soporta múltiples formatos de archivo, como **CSV, JSON, Parquet, ORC y Avro**.

Por ejemplo, para leer un archivo CSV:

```
df_csv = spark.read.csv("ruta/datos.csv", header=True, inferSchema=True)
df_csv.show()
```

Creación desde archivos JSON

También es posible crear un DataFrame leyendo archivos en formato **JSON**:

```
# Leer un archivo JSONdf_json =  
spark.read.json("ruta/a1/archivo.json")
```

```
import org.apache.spark.{SparkConf, SparkContext}  
import org.apache.spark.sql.{SparkSession, DataFrame}  
import org.apache.spark.sql.functions._  
  
object ConvertDFToJson extends App {  
  
  //Creating SparkSession  
  lazy val conf = new SparkConf().setAppName("ddv-parquet-generator").set("spark.default.parallelism", "2")  
    .setIfMissing("spark.master", "local[*]")  
  lazy val sparkSession = SparkSession.builder().config(conf).getOrCreate()  
  lazy val sparkContext: SparkContext = sparkSession.sparkContext  
  import sparkSession.implicits._  
  
  //Creating Input DataFrame  
  val inputDF = Seq(("Anamika", 30, "Delhi", Map("First" -> "ABC")), ("Ananya", 25, "Delhi", Map("Second" -> "XYZ")), ("Pritam", 12, "U.P.", Map("Third" -> "MNO"))  
    .toDF( colNames = "Name", "Age", "City", "Data")  
  inputDF.show( truncate = false)  
  
  //For the last column, To change the key as City not Data :  
  //Reshaping the Columns using pivot  
  val transformedDF = inputDF.groupBy( col = "Name", cols = "Age", "City").pivot( pivotColumn = "city").agg(first( columnName = "Data"))  
  transformedDF.show( truncate = false)  
  
  //Now make it by using toJson and collect it.  
  //Converting into json as expected output and collect it to print.  
  val jsonArray = transformedDF.toJSON.collect  
  jsonArray.foreach(println)  
}
```

Creación desde archivos Parquet

El formato **Parquet**, que ofrece almacenamiento eficiente y optimizado para consultas analíticas, también puede ser utilizado para crear un DataFrame:

```
df_parquet = spark.read.parquet("ruta/datos.parquet")  
df_parquet.show()
```

Creación desde bases de datos

Otra opción para crear un DataFrame es utilizando fuentes de datos externas, como bases de datos relacionales. Spark SQL permite conectarse a bases de datos mediante el protocolo **JDBC**.

```
df_jdbc = spark.read \  
    .format("jdbc") \  
    .option("url", "jdbc:mysql://localhost:3306/basededatos") \  
    .option("dbtable", "tabla") \  
    .option("user", "usuario") \  
    .option("password", "contraseña") \  
    .load()
```


Creación desde consultas SQL

Consultas sobre tablas existentes

Spark SQL permite crear DataFrames a partir de consultas SQL sobre otras tablas o DataFrames existentes:

```
# Registrar un DataFrame como vista temporal
df.createOrReplaceTempView("personas")

# Crear un nuevo DataFrame a partir de una consulta SQL
df_mayores = spark.sql("SELECT * FROM personas WHERE edad > 30")
```

Métodos de exploración de un DataFrame

Una vez creado un **DataFrame** en Apache Spark, es fundamental poder explorarlo para conocer su estructura, el tipo de datos que contiene y realizar un análisis preliminar antes de aplicar transformaciones o consultas avanzadas. Spark SQL ofrece múltiples métodos para explorar y examinar los DataFrames de manera eficiente.

Visualización de datos con show()

Método show()

Uno de los métodos más utilizados es **show()**, que permite visualizar las primeras filas del DataFrame en la consola. Por defecto, muestra las primeras 20 filas, pero es posible especificar un número distinto:

```
# Mostrar las primeras 20 filas
df.show()

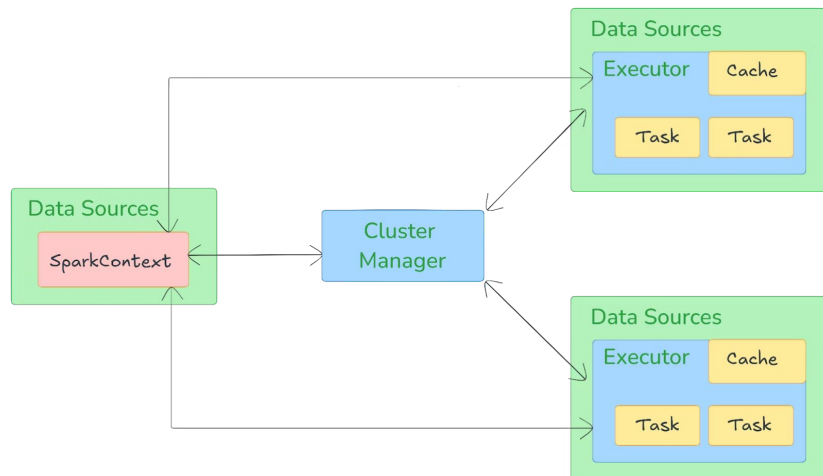
# Mostrar las primeras 5 filas
df.show(5)

# Mostrar todas las filas sin truncar el contenido
df.show(truncate=False)
```

Estructura del DataFrame

Otra función importante es **printSchema()**, que muestra la estructura del DataFrame, incluyendo los nombres de las columnas y los tipos de datos asociados. Esta función es útil para verificar que los datos han sido leídos correctamente y que los tipos son los esperados:

```
# Mostrar el esquema del DataFrame  
df.printSchema()
```



Nombres de columnas

Método `columns`

El método **`columns`** permite obtener una lista con los nombres de las columnas del DataFrame:

```
# Obtener los nombres de las columnas
columnas = df.columns
print(columnas)
```

Resumen estadístico

Para obtener un resumen estadístico de las columnas numéricas, Spark SQL ofrece el método **describe()**. Este método calcula métricas como el conteo, la media, la desviación estándar, el valor mínimo y máximo:

```
# Obtener estadísticas descriptivas  
df.describe().show()
```

Información detallada

Método `summary()`

Si se desea obtener información más detallada sobre cada columna, como la cantidad de valores nulos, los tipos de datos y las estadísticas básicas, se puede utilizar el método **`summary()`**:

```
# Obtener un resumen completo  
df.summary().show()
```

Conteo de registros

El método **count()** permite conocer la cantidad total de registros que contiene el DataFrame:

```
# Contar el número de filas
num_filas = df.count()
print(f"El DataFrame contiene {num_filas} filas")
```


Primeras filas

Otra función útil es **head()**, que devuelve la primera fila del DataFrame como un objeto Row. También se puede utilizar **first()**, que cumple la misma función:

```
primera_fila = df.head()
print(primera_fila)
```

Valores únicos

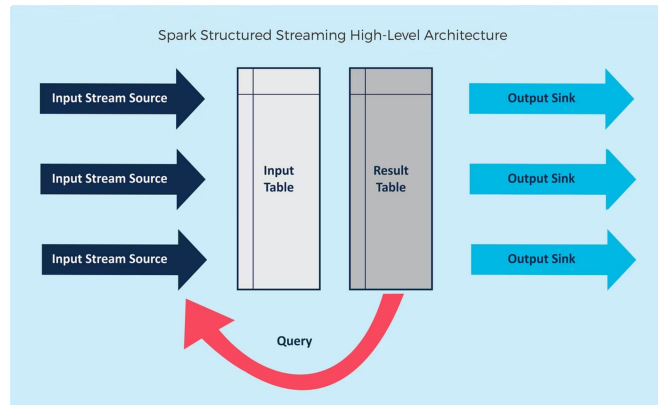
Método **distinct()**

Para obtener los valores únicos de una columna específica, se puede utilizar el método **distinct()** combinado con **select()**:

```
# Obtener valores únicos de una columna  
df.select("departamento").distinct().show()
```

Uso de DataFrames para consultas

Uno de los mayores beneficios de los **DataFrames** en Apache Spark es la posibilidad de realizar consultas sobre los datos de forma eficiente y utilizando un lenguaje familiar como **SQL** o funciones de la API de DataFrame. Esto facilita la exploración, transformación y análisis de grandes volúmenes de datos estructurados.



Selección de columnas

Método select()

Una de las formas más directas de consultar un DataFrame es utilizando métodos de filtrado y selección de columnas.

Por ejemplo, para seleccionar una columna específica, se puede utilizar el método **select()**:

```
# Seleccionar una columna
df.select("nombre").show()

# Seleccionar múltiples columnas
df.select("nombre", "edad", "salario").show()
```

Transformaciones encadenadas

También es posible aplicar múltiples transformaciones encadenadas. Por ejemplo, para seleccionar registros donde la edad sea mayor a 30 y ordenar los resultados por nombre:

```
# Filtrar y ordenar
df.filter(df.edad > 30).orderBy("nombre").show()
```

Transformations:	
<code>map(f : T -> U):</code>	<code>RDD<T> -> RDD<U></code>
<code>flatMap(f: T -> Seq(U)):</code>	<code>RDD<T> -> RDD<U></code>
<code>filter(f:T->Bool):</code>	<code>RDD<T> -> RDD<T></code>
<code>groupByKey():</code>	<code>RDD<(K,V)> -> RDD<(K,Seq[V])></code>
<code>reduceByKey(F:(V,V)-> V):</code>	<code>RDD<(K,V)> -> RDD<(K,V)></code>
<code>union():</code>	<code>(RDD<T>,RDD<T>) -> RDD<T></code>
<code>join():</code>	<code>(RDD<(K,V)>,RDD<(K,W)>) -> RDD<(K,(V,W))></code>
<code>cogroup():</code>	<code>(RDD<(K,V)>,RDD<(K,W)>)-> RDD<(K,(Seq<V>,Seq<W>))></code>
<code>crossProduct():</code>	<code>(RDD<T>,RDD<U>) -> RDD<(T,U)></code>

Actions:	
<code>count():</code>	<code>RDD<T> -> Long</code>
<code>collect():</code>	<code>RDD<T> -> Seq<T></code>
<code>reduce(f:(T,T)->T):</code>	<code>RDD<T> -> T</code>
<code>save(path:String):</code>	Outputs RDD to a storage system e.g., HDFS

Vistas temporales

Otra funcionalidad poderosa es la posibilidad de registrar un DataFrame como una **vista temporal (Temporary View)** y ejecutar consultas SQL estándar sobre él. Esto se hace con el método **createOrReplaceTempView()**:

```
# Crear una vista temporal  
df.createOrReplaceTempView("empleados")
```

Consultas SQL

Una vez registrada la vista, es posible ejecutar consultas SQL utilizando el método **spark.sql()**:

```
resultado = spark.sql("SELECT Nombre, Edad FROM empleados WHERE Edad > 30")  
resultado.show()
```

Agregaciones

Funciones de agregación

Además de consultas simples, se pueden realizar operaciones más complejas como agregaciones, uniones y combinaciones de DataFrames. Por ejemplo, para calcular la edad promedio de los empleados:

```
# Calcular la edad promedio
from pyspark.sql.functions import avg
df.select(avg("edad").alias("edad_promedio")).show()

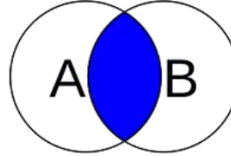
# Usando SQL
spark.sql("SELECT AVG(edad) AS edad_promedio FROM empleados").show()
```


Uniones de DataFrames

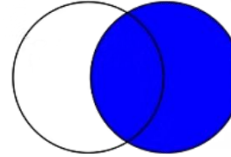
También es posible unir dos DataFrames utilizando el método **join()**, especificando la columna clave para la unión:

```
# Unir dos DataFrames  
df_empleados.join(df_departamentos,df_empleados.id_dept == df_departamentos.id, "inner").show()
```

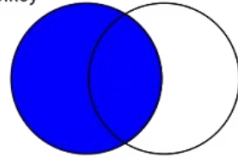
```
SELECT <fields>
FROM TableA A
INNER JOIN TableB B
ON A.key = B.key
```



```
SELECT <fields>
FROM TableA A
RIGHT JOIN TableB B
ON A.key = B.key
```

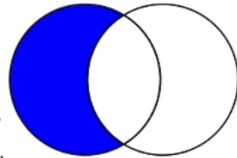


```
SELECT <fields>
FROM TableA A
LEFT JOIN TableB B
ON A.key = B.key
```

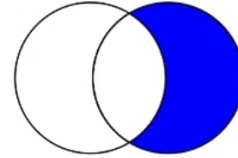


SQL JOINS

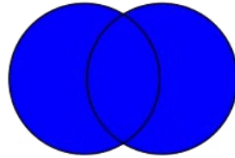
```
SELECT <fields>
FROM TableA A
LEFT JOIN TableB B
ON A.key = B.key
WHERE B.key IS NULL
```



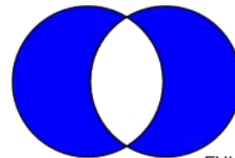
```
SELECT <fields>
FROM TableA A
RIGHT JOIN TableB B
ON A.key = B.key
WHERE A.key IS NULL
```



```
SELECT <fields>
FROM TableA A
FULL OUTER JOIN TableB B
ON A.key = B.key
```



```
SELECT <fields>
FROM TableA A
FULL OUTER JOIN TableB B
ON A.key = B.key
WHERE A.key IS NULL
OR B.key IS NULL
```

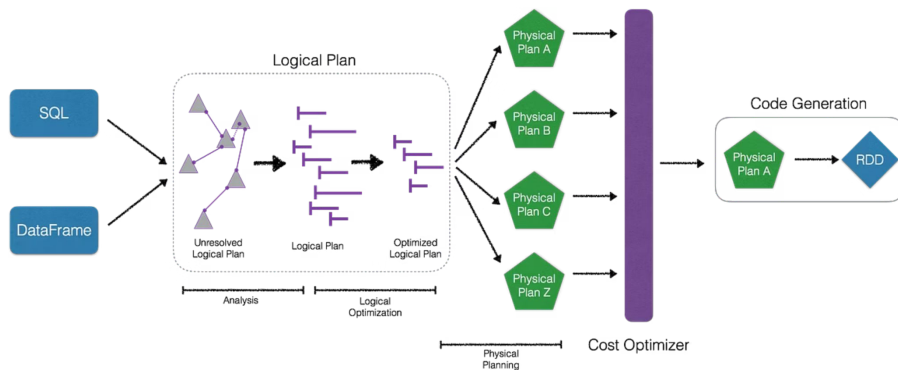


Expresiones SQL

Método selectExpr()

Otra operación común es el uso de expresiones SQL dentro de las funciones de DataFrame mediante **selectExpr()**, que permite aplicar expresiones SQL sin necesidad de registrar una vista:

```
# Usar expresiones SQL en selectExpr
df.selectExpr( "nombre", "edad", "salario * 1.1 AS nuevo_salario", "CASE WHEN edad > 40 THEN
'Senior' ELSE 'Junior' END AS categoría").show()
```



Optimización de consultas

Uno de los elementos que diferencian a **Apache Spark** de otras plataformas de procesamiento de datos es su capacidad para optimizar las consultas de forma automática y eficiente. Spark SQL incorpora dos proyectos clave para este propósito: **Catalyst** y **Tungsten**.

Catalyst Optimizer



El **Catalyst Optimizer** es el motor de optimización de consultas de Spark SQL. Su objetivo principal es transformar las consultas lógicas realizadas por el usuario en planes físicos optimizados que puedan ejecutarse de manera eficiente en el clúster.

Etapas de optimización de Catalyst

Análisis lógico

Se valida la estructura de la consulta, se resuelven las referencias a columnas y se aplican reglas de simplificación.

Optimización lógica

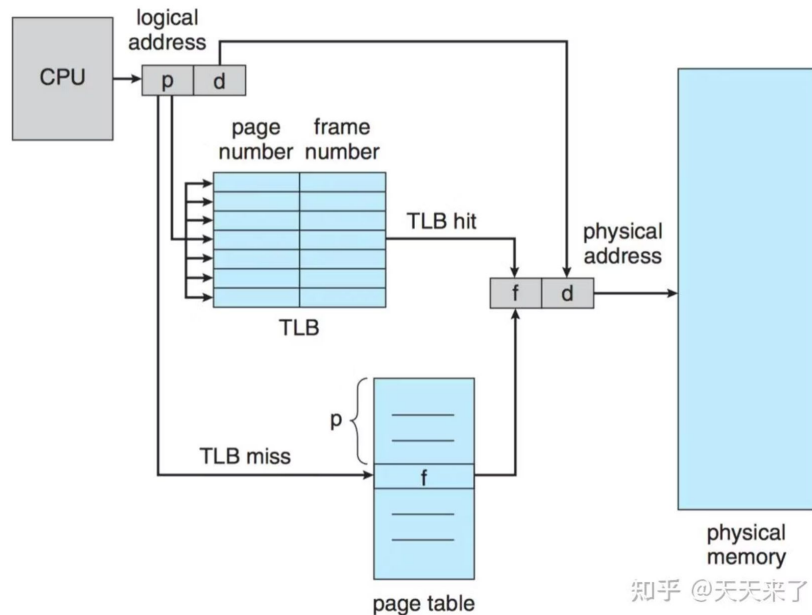
Se aplican reglas como la eliminación de columnas no utilizadas, el reordenamiento de filtros y la combinación de proyecciones.

Generación de plan físico

Se define cómo se ejecutará la consulta en el clúster, incluyendo detalles sobre las operaciones que se ejecutarán en cada nodo.

Proyecto Tungsten

Además de Catalyst, Spark SQL incorpora el proyecto **Tungsten**, que se centra en la optimización del uso de recursos a nivel bajo. Tungsten mejora la eficiencia de ejecución mediante técnicas como la gestión eficiente de la memoria, la generación de código bytecode y la ejecución vectorizada.



Beneficios de Tungsten



Optimización de memoria

Implementa una gestión de memoria fuera de la JVM (off-heap memory), reduciendo la latencia y mejorando la estabilidad.



Generación de código

Genera código específico para cada consulta, eliminando sobrecargas y mejorando el rendimiento de ejecución.



Ejecución vectorizada

Procesa múltiples registros en una sola operación, aprovechando las capacidades de los procesadores modernos.

Transparencia para el usuario

Estas optimizaciones son transparentes para el usuario. No es necesario escribir código específico para beneficiarse de ellas, ya que se aplican automáticamente cuando se ejecutan consultas sobre DataFrames o vistas temporales.

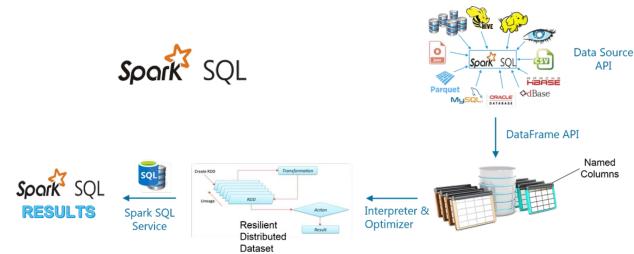


Figure: The flow diagram represents a Spark SQL process using all the four libraries in sequence

Rendimiento comparativo

La combinación de Catalyst y Tungsten permite que Spark SQL ofrezca un rendimiento comparable e incluso superior al de sistemas tradicionales de bases de datos, mientras mantiene la flexibilidad del procesamiento distribuido.



Conclusión

Procesamiento eficiente

Catalyst y Tungsten son dos pilares fundamentales que permiten a Spark SQL ejecutar consultas de manera eficiente y escalable, aprovechando al máximo los recursos del clúster y mejorando la experiencia del usuario.

Base sólida

El conocimiento adquirido en este manual proporciona una base sólida para que los estudiantes y profesionales puedan implementar soluciones de análisis de datos estructurados utilizando Apache Spark, integrando datos de diversas fuentes y aplicando técnicas avanzadas de consulta y transformación.



Live Coding

¿En qué consistirá la Demo?

Exploraremos consultas avanzadas sobre grandes volúmenes de datos estructurados y veremos cómo Spark las optimiza automáticamente.

1. Filtrado y ordenamiento encadenado sobre un DataFrame.
2. Agregaciones y uso de funciones como `avg()`, `count()`, `sum()`.
3. Uniones internas y externas (`inner join`, `left join`).
4. Uso de `selectExpr` para aplicar expresiones SQL directamente.
5. Visualización del plan de ejecución y optimización automática con Catalyst.



Ejercicio

Consultas optimizadas en Spark SQL

Consultas optimizadas en Spark SQL

Contexto: 🙌

Aplicarás técnicas avanzadas de consulta y optimización en Spark SQL para procesar datos estructurados de forma eficiente.

Consigna: 📝

1. Carga un dataset en formato Parquet o CSV como DataFrame.
 - a. Realiza al menos tres consultas avanzadas que incluyan:
 - a. Filtros y ordenamientos encadenados.
 - b. Agregaciones con funciones estadísticas.
 - c. Una unión entre DataFrames.
 - d. Aplica expresiones SQL con selectExpr.
2. Observa el plan de ejecución (explain()) y analiza cómo Catalyst lo optimiza.

Paso a paso: ⚙️

1. Inicia la sesión Spark y carga los datos.
2. Ejecuta las consultas y guarda los resultados.
3. Usa explain() para visualizar el plan lógico y físico.
4. Documenta cómo Catalyst y Tungsten intervienen en la optimización.

Tiempo 🕒: 45 Minutos

¿Alguna consulta?





¡Ponte a prueba!

Momento de ejercitación

*Te invitamos a aprovechar esta última sección del espacio sincrónico para realizar de manera individual las **actividades disponibles en la plataforma**. Estas propuestas son clave para afianzar lo trabajado y **forman parte obligatoria del recorrido de aprendizaje**.*

👉 **Análisis de caso** ————— 👉 **Selección Múltiple**

👉 **Comprensión lectora**

Si al resolverlas surge alguna duda, compartela o tráela al próximo encuentro sincrónico.

< ¡Muchas gracias! >

